

React JS

B. Vijay Kumar

React JS : Content

- History of Front – end libraries
- Key differentiators (Virtual DOM, one – way binding)
- React Components
- What is JSX?
- props
- state
- hooks
- events and effects
- fetching data from API using fetch
- form validations
- React Router
- Building and Deploying react application.

History

ReactJS is a JavaScript library used for building user interfaces (UIs), especially for single-page applications (SPAs).

It was developed by Facebook in 2013 and is now maintained by Meta (formerly Facebook) and a large open-source community.

History of Front-End Libraries:

- **Early Days: Static Webpages**

- 1990s: Websites were primarily static, created using HTML, CSS, and some JavaScript for interactivity.
- Developers began looking for tools to simplify scripting and improve code organization.

- **The Advent of JavaScript Libraries**

2000s: JavaScript libraries like jQuery gained popularity for simplifying DOM manipulation, AJAX requests, and event handling.

- Strength: Made JavaScript easier to use across different browsers.
- Limitations: jQuery wasn't designed for building complex, interactive applications. Codebases could become messy as applications grew.

History

- **Emergence of Front-End Frameworks**

- **Backbone.js (2010):** Introduced a structure with Models, Views, and Collections. Allowed developers to create single-page applications (SPAs).
- **AngularJS (2010):** Developed by Google, AngularJS introduced two-way data binding and dependency injection. Revolutionized dynamic views in web applications.
- **Knockout.js (2010):** Focused on data binding and the Model-View-ViewModel (MVVM) pattern.

Introduction

ReactJS: Key differentiators (Virtual DOM, one – way binding)

- Created by Jordan Walke, a Facebook engineer, React was first deployed on Facebook's newsfeed and later on Instagram.
- ReactJS is a JavaScript library used for building user interfaces (UIs), especially for single-page applications (SPAs).
- React introduced a component-based architecture, making it easier to reuse and maintain code.

Introduction

Features:

- **Component-Based Architecture:**

1. React applications are built using components, which are reusable, self-contained pieces of code.
2. Each component manages its own logic and state, making development more modular and easier to maintain.

- **Virtual DOM:**

1. React uses a virtual DOM (Document Object Model) to optimize performance.
2. When changes are made, React updates only the parts of the DOM that have changed, instead of reloading the entire page.

Introduction

Features:(Continued...)

- **Declarative Syntax:**

1. React allows developers to design views by declaring how the UI should look based on the application's state.
2. This makes the code easier to read and debug.

- **JSX (JavaScript XML):**

React uses JSX, which combines HTML and JavaScript syntax in one file, making it easier to design components.

- **Unidirectional Data Flow:**

Data flows in one direction, making it easier to understand and debug applications.

Introduction

Benefits:

- **Performance:** Virtual DOM ensures faster updates and rendering.
- **Reusable Components:** Saves development time and ensures consistency.
- **Cross-Platform Development:** With tools like React Native, you can build mobile apps using the same React principles.
- **Strong Community Support:** Extensive documentation, tutorials, and third-party libraries are available.

First Project

To create a new React project, follow these steps:

Step 1:

We need to install Node.js and npm(Node package manager)

Download Link: <https://nodejs.org/en/download/prebuilt-installer>

Download Node.js®

Download Node.js the way you want.

Package Manager **Prebuilt Installer** Prebuilt Binaries Source Code

I want the **v20.15.1 (LTS)** version of Node.js for **Windows** running **x64**

 **Download Node.js v20.15.1**

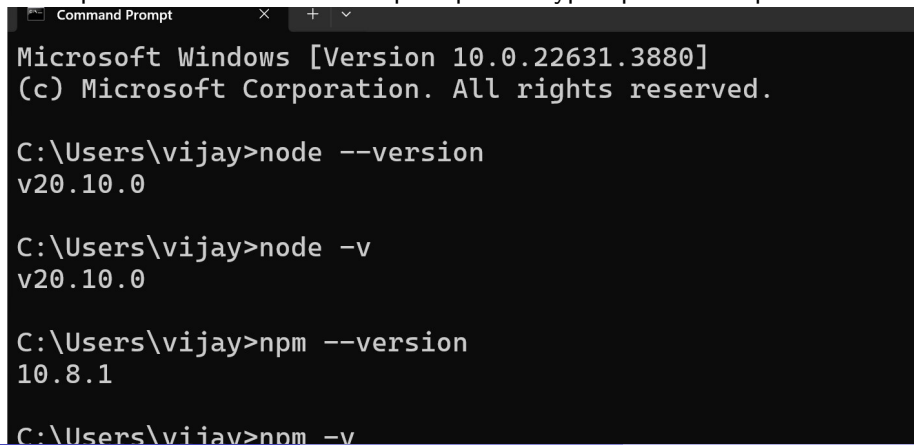
Node.js includes npm (10.7.0) as well.

First Project

Step 2:

Verify Installation

- Node: Go to the command prompt and type `node -v` or `node --version`
- npm: Go to the command prompt and type `npm -v` or `npm --version`



```
Command Prompt
Microsoft Windows [Version 10.0.22631.3880]
(c) Microsoft Corporation. All rights reserved.

C:\Users\vijay>node --version
v20.10.0

C:\Users\vijay>node -v
v20.10.0

C:\Users\vijay>npm --version
10.8.1

C:\Users\vijay>npm -v
```

First Project

Step 3: Create React App (CRA)

```
npx create-react-app first-react-app
```

Step 4: Navigate to app directory

```
cd first-react-app
```

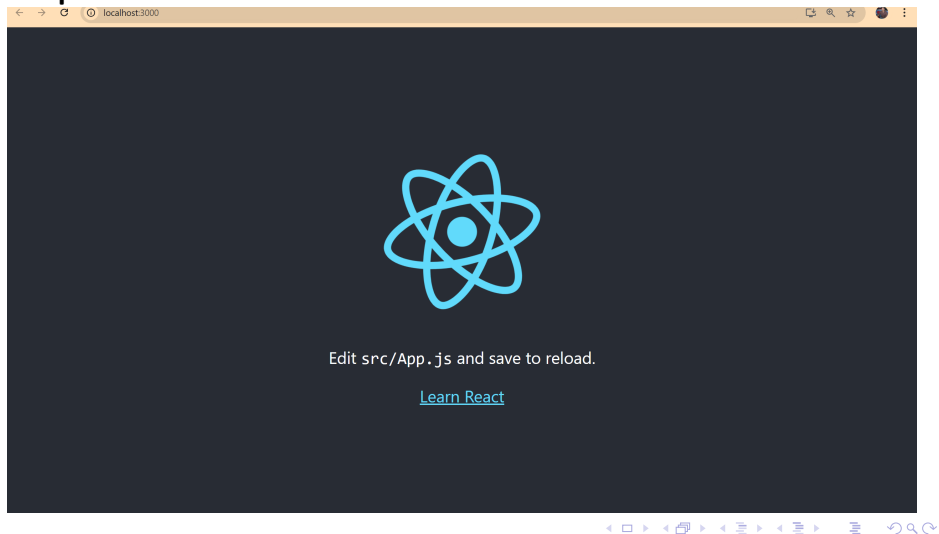
Step 5: Start the development server

```
npm start
```

Note: This command starts a development server and opens the default React app in your browser at <http://localhost:3000/>.

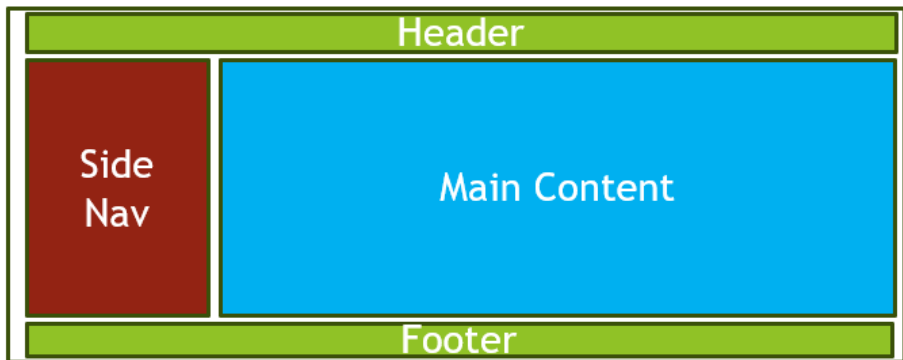
First Project

Output:



React Components

- In React, Components are called as a part User Interface(UI)
- Components are independent and reusable bits of code.



In the above diagram, Each Header, Side Nav, Main Content and Footer are Built as components.

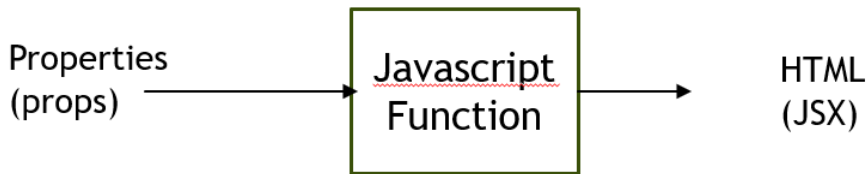
React Components

Components come in two types:

- Function components
- Class components

Function components

These are simple JavaScript functions that accept props as an argument and return React elements (JSX).



React Components

Features:

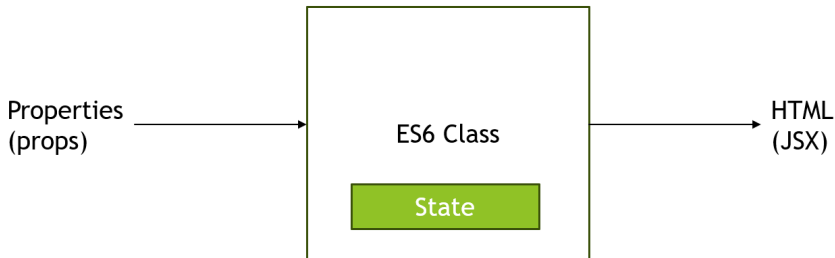
- Stateless by default but can manage state using React Hooks like `useState` or `useEffect`.
- Easier to write and test.
- Lightweight and preferred for most use cases in modern React.

```
function MyGreeting(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}  
  
export default MyGreeting
```

React Components

Class components:

In react class component is a class that extends `React.Component` and gives access to `React.Component` class functions.



React Components

Features:

- Can manage state and have lifecycle methods (e.g., `componentDidMount`, `componentDidUpdate`).
- Heavier than functional components.
- Gradually being replaced by functional components with hooks.

```
import React,{Component} from "react";  
class MyGreeting extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}!</h1>;  
  }  
}  
export default MyGreeting
```

Key Differences of Components

Functional Components	Class Components
Simple Functions	More feature rich
Use Func Components as much as possible	Main its own private data –state
Absence of 'this' keyword	Complex UI Logic
Solution without using state	Provide Life cycle hooks
Stateless/Dumb/Presentational	Stateful/Smart/Container
Mainly responsible for the UI	Mainly responsible for the Logic

Create a Component

Here are the steps to create a Component, Which are similar for class component:

- create a function component with the name FunctionGreeting.js in Components folder
- Write the following code in it

```
function MyGreeting_F(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}  
export default MyGreeting_F
```

- import the component in App.js as below:

```
import MyGreeting_F from './Components/  
FunctionGreeting';
```

- Use it in app function as below:

```
<MyGreeting_F name="Vijay"/>
```

JSX

JSX: Stands for (JavaScript XML) is a syntax extension for JavaScript

- Commonly used with React to describe the structure of the user interface (UI).
- JSX allows you to write HTML-like code directly within JavaScript, making it easier to visualize and compose the UI.

Example:

```
function Welcome(props) {  
  return (  
    <div>  
      <h1>Welcome, {props.name}!</h1>  
      <p>We are glad to have you here.</p>  
    </div>  
  );  
}  
export default Welcome;
```

Props

Props:

- Props (short for "properties") are a mechanism in React used to pass data from one component to another, typically from a parent component to its child components.
- They enable components to be dynamic and reusable.

Example:

```
function Greeting(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```

Props

Key Features of Props:

- **Read-Only:** Props are immutable, meaning they cannot be modified by the child component receiving them.
- **Unidirectional Data Flow:** Data flows in a single direction, from parent to child.
- **Customizable Components:** Props allow you to customize components based on the data passed to them.

State

State: state is a built-in object that allows components to store and manage dynamic data that can change over time. It helps a component re-render and update the UI whenever the state changes.

Features of State in React:

- **Component-Specific:** Each component can have its own state, which is independent of other components.
- **Dynamic and Mutable:** The state can be updated over time, usually in response to user actions or other events.
- **Reacts to Changes:** When the state is updated, the component re-renders to reflect the new state in the UI.

Note: Updating state in components will be done as below:

- **Updating State: In Class Components:** Use `this.setState()` to update the state.
- **In Functional Components:** Use the setter function (e.g., `setCount`) from the `useState` hook.

Hooks

Hooks:

- These are the functions that allow you to use React state and lifecycle features in functional components.
- Introduced in React 16.8, hooks make it possible to write components without needing to convert them to class components for state or lifecycle management.

These are commonly used hooks in React:

- useState
- useEffect
- useContext
- useReducer
- useRef
- useMemo
- useCallback

Hooks

useState: Adds state to functional components. **Example:**

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
Increment</button>
    </div>
  );
}

export default Counter
```

Hooks

useEffect: Manages side effects such as data fetching, subscriptions, or manually changing the DOM. **Example:**

```
import React, { useState, useEffect } from 'react';

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds((prev) => prev + 1);
    }, 1000);

    return () => clearInterval(interval); // Cleanup on
    unmount
  }, []);

  return <div>Seconds: {seconds}</div>;
}

export default Timer
```

Hooks

useContext: Consumes values from a React context. **Example:**

```
import React, { useContext, createContext } from 'react';

const ThemeContext = createContext('light');

function ThemedComponent() {
  const theme = useContext(ThemeContext);

  return <div>Current theme: {theme}</div>;
}

export default ThemedComponent
```

Hooks

useReducer: Manages complex state logic, similar to useState but with reducers.

Example:

```
import React, { useReducer } from 'react';

const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    default:
      return state;
  }
}
```

Hooks

useReducer:(Continues...)

```
function Counter() {
  const [state, dispatch] = useReducer(reducer,
    initialState);

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: '
increment' })}></button>
      <button onClick={() => dispatch({ type: '
decrement' })}></button>
    </div>
  );
}

export default Counter
```

Hooks

useRef:Accesses or modifies a DOM element directly or stores a mutable value. **Example:**

```
import React, { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef();

  const focusInput = () => {
    inputRef.current.focus();
  };

  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={focusInput}>Focus Input</button>
    </div>
  );
}
```

Hooks

useMemo: Memoizes expensive computations to improve performance.

Example:

```
import React, { useMemo, useState } from 'react';

function ExpensiveComputation({ num }) {
  const compute = useMemo(() => {
    console.log('Calculating...');
    return num * 2;
  }, [num]);

  return <div>Computed value: {compute}</div>;
}

export default ExpensiveComputation
```

Hooks

useCallback: Memoizes functions to prevent unnecessary re-creations.

Example:

```
import React, { useState, useCallback } from 'react';

function CallbackCounter() {
  const [count, setCount] = useState(0);

  const increment = useCallback(() => {
    setCount((prevCount) => prevCount + 1);
  }, []);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>
    </div>
  );
}

export default CallbackCounter
```


Events and Effects

Events are handled similarly to how they are in plain JavaScript but with a React-specific syntax.

Points to remember to implement Events:

- **CamelCase Syntax:** Event names are written in camelCase (e.g., `onClick` instead of `onclick`).
- **JSX Function References:** You pass a function reference (not a string or a function call) as the event handler.
- **Synthetic Events:** React wraps native DOM events in its own `SyntheticEvent` for cross-browser compatibility.
- **Default Behavior:** To prevent default behavior (like form submission), you call `event.preventDefault()`

Events and Effects

Commonly used React Events are:

- **Mouse Events:** onClick, onDoubleClick, onMouseEnter, onMouseLeave
- **Keyboard Events:** onKeyDown, onKeyPress, onKeyUp
- **Form Events:** onChange, onSubmit, onFocus, onBlur
- **Window Events:** onLoad, onResize, onScroll

Events and Effects

Example1: Handling mouse event

```
function Mouse_Event() {  
  const handleClick = () => {  
    alert("Button clicked!");  
  };  
  
  return <button onClick={handleClick}>Click Me</button>;  
}  
export default Mouse_Event
```

Events and Effects

Example2: Handling form event

```
function Form_Event() {  
  const handleSubmit = (event) => {  
    event.preventDefault();  
    console.log("Form submitted!");  
  };  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <input type="text" placeholder="Enter text" />  
      <button type="submit">Submit</button>  
    </form>  
  );  
}  
export default Form_Event
```

Events and Effects

Example3: Handling keyboard event

```
function Form_Event() {  
  const handleSubmit = (event) => {  
    event.preventDefault();  
    console.log("Form submitted!");  
  };  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <input type="text" placeholder="Enter text" />  
      <button type="submit">Submit</button>  
    </form>  
  );  
}  
  
export default Form_Event
```

Events and Effects

Example4: Passing Arguments to mouse event

```
function Mouse_Event_Arguments() {  
  const handleClick = (name) => {  
    alert('Hello, ${name}!');  
  };  
  
  return <button onClick={() => handleClick("Vijay")}>  
    Greeting</button>;  
}  
export default Mouse_Event_Arguments
```

Events and Effects

Effects: effects are functions that allow you to handle side effects in function components. Side effects include operations like:

- Fetching data from an API.
- Subscribing to events.
- Updating the DOM manually.
- Interacting with browser APIs.

Note: The easy way to handle effects in React is through the `useEffect`

Events and Effects

useEffect: useEffect lets us perform side effects in function components. It runs after the render and can re-run based on specified dependencies.

Syntax:

```
useEffect(effectFunction, dependencyArray);
```

- effectFunction: A function containing the side-effect logic.
- dependencyArray: An array of dependencies that determine when the effect should re-run. If omitted, the effect runs

Events and Effects

useEffect:(Example)

```
import React, { useState, useEffect } from "react";

const Effect_Example = () => {
  const [data, setData] = useState(null); // State to hold
    fetched data
  const [loading, setLoading] = useState(true); // State to
    handle loading state

  // useEffect to fetch data
  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch("https://
jsonplaceholder.typicode.com/posts/1");
        const result = await response.json();
        setData(result); // Update the state with fetched
data
        setLoading(false); // Set loading to false
      } catch (error) {
        console.log(error);
      }
    };
    fetchData();
  }, []);
}
```

Events and Effects

useEffect:(Example)(Continued..)

```
    } catch (error) {  
      console.error("Error fetching data:", error);  
    }  
  };  
  
  fetchData(); // Call the fetchData function  
}, []); // Empty dependency array means this effect runs  
        once when the component mounts  
  
// JSX  
return (  
  <div>  
    {loading ? (  
      <p>Loading...</p>  
    ) : (  
      <div>  
        <h1>{data.title}</h1>  
        <p>{data.body}</p>  
      </div>  
    )  
  }  
)
```

Events and Effects

useEffect:(Example)(Continued..)

```
        </div>
      )}
    </div>
  );
};

export default Effect_Example;
```

FetchData

Step 1:Set Up the Express API

```
// server.js (Express API)
const express = require('express');
const app = express();
const cors = require('cors');

app.use(cors()); // Allow cross-origin requests
app.use(express.json());

app.get('/api/data', (req, res) => {
  res.json({ message: 'Hello from the server!' });
});

app.listen(5000, () => {
  console.log('Server running on port 5000');
});
```

FetchData

Step 2: Create a React Project

Step 3: Fetch Data in React

```
import React, { useState, useEffect } from 'react';

function App() {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch('http://localhost:5000/api/data')
      .then((response) => response.json())
      .then((data) => setData(data))
      .catch((error) => console.error('Error fetching data
: ', error));
  }, []);
```

FetchData

Step 3:(Continued...)Fetch Data in React

```
return (  
  <div>  
    <h1>Data from API:</h1>  
    {data ? <p>{data.message}</p> : <p>Loading...</p>}  
  </div>  
) ;  
}  
export default App;
```

Step 4:Start both the servers

- Start Express: node server.js
- Start React: npm start

Questions

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ 🔍 ↺